

Due Date: Wednesday, **August 6 @ 11:59 PM PDT** (No late submissions!)

What you will build

1. A **16-bit combinational barrel shifter** that can *logically shift* or *rotate* left / right by an arbitrary amount k (0–15) in **one clock tick**.
2. **Input interface:** the raw 16-bit word comes from **SW15–SW0**.
3. **Control interface:** the four on-board push-buttons (BTNU, BTND, BTNL, BTNR) are debounced and latched to behave like extra toggle-switches that provide:
4. **DIR** (Left = 0, Right = 1)
5. **ROT** (Logical = 0, Rotate = 1)
6. **SHAMT[1:0]** – the two LSBs of the 4-bit shift amount
7. **BTN_center (BTNC)** is reserved for **asynchronous reset**
8. **SHAMT[3:2]** come from a tiny up-counter that increments once per BTNC press so you can demo all 16 positions without giving up slide-switches.
9. **Output interface:** the 16-bit result is shown as four hexadecimal nibbles on the **right-most four digits (AN0–AN3)** of the board's 8-digit 7-segment display. The remaining digits (AN4–AN7) are blanked.
10. A **fixed clock-divider** slows the 100 MHz system clock to ~ 2 Hz for the shifter demo and ~ 1 kHz for display scanning — fast enough for the display, slow enough for human eyes to track shifts.
11. **Optional debug LEDs:** display DIR, ROT, and SHAMT on LED7–LED0.

How many 2-to-1 multiplexers does it take?

A log-stage barrel shifter uses $n \cdot \log_2 n$ 2-input MUXes. For $n = 16$:

- $\log_2 16 = 4$ stages
- **16 MUXes per stage $\times 4 = 64$ MUX2 $\times$ 1** for one-direction logical shifting.
- Adding both directions **and** rotation can reuse most wires, but if you build two independent trees the worst-case budget is **128 MUX2 $\times$ 1** — still negligible in an Artix-7 FPGA.

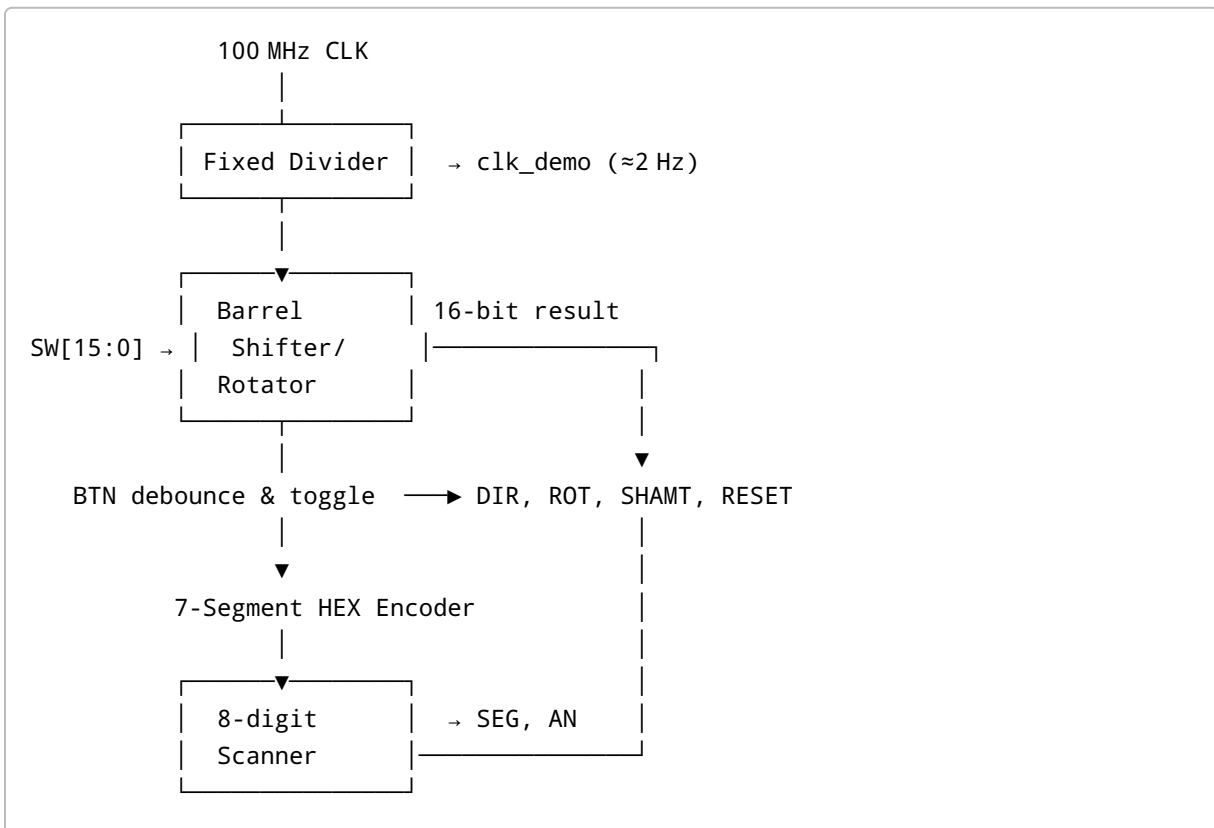
Lab Objectives (plain English)

- See how a tree of tiny 2 $\times$ 1 MUXes can instantly permute a 16-bit word.
 - Practice **push-button debouncing & edge-to-toggle logic**.
 - Refresh your 7-segment display skills from Lab 6, now driving four hex digits.
 - Gain intuition for hardware cost vs. functionality trade-offs (64 vs 128 MUXes).
-

Hardware Pin Mapping (Nexys A7-100T)

Signal	Port/Pin	Description
CLK	100 MHz	On-board master clock
SW15-SW0	SW[15:0]	16-bit input word
BTNC	BTN0	Reset (active low)
BTNU, BTND, BTNL, BTNR	BTN[4:1]	Debounced control toggles (DIR, ROT, SHAMT[1:0])
LED7-LED0	LED[7:0]	Debug:
AN7-AN0	AN[7:0]	Digit enables (we use AN0-AN3)
SEG6-SEG0	SEG[6:0]	Seven-segment segments

Top-Level Block Diagram



(Exact artwork optional in your report.)

Step-by-Step Implementation

1. **clock_divider_fixed.v**\ Parameter: `DIV_VALUE = 26'd50_000_000` gives ~1 Hz; use two cascaded counters to obtain both ~1 kHz (for scanning) and ~2 Hz (for visible shifting).
2. **debounce_toggle.v**\ Inputs: `btn_raw`, `clk_1kHz` \ Outputs: `btn_toggle` (latched high/low on every clean edge).
3. **barrel_shifter16.v**\ Inputs: `data_in[15:0]`, `shamt[3:0]`, `dir`, `rotate` \ *Combinational* output `data_out[15:0]`. Each stage is a generate-for loop that synthesises 16 LUT6s.
4. **shamt_counter.v**\ A 2-bit free-run counter that advances on every BTNC press to provide `shamt[3:2]`.
5. **hex_to_7seg.v**\ Same lookup table you wrote in Lab 6 -- now reused four times.
6. **seg7_scan8.v**\ 8-digit scanner (refresh ≈ 1 kHz). Drive AN0-AN3, leave AN4-AN7 off.
7. **top_lab7.v**
8. Instantiate all modules.
9. Wire slide-switches, debounced buttons, LEDs, 7-segment pins.
10. Use a constant `assign result_word = barrel_out;`.

Testing & Simulation

Module	Testbench Goals
<code>barrel_shifter16_tb</code>	Exhaustively sweep <code>shamt</code> , <code>dir</code> , <code>rotate</code> on a few patterns (0xA5A5, 0x8001, etc.).
<code>debounce_toggle_tb</code>	Verify button bounces are filtered (inject random glitches).
<code>seg7_scan8_tb</code>	Check 1-of-8 enable sequence and segment codes.

Waveform snapshots must appear in your PDF report.

Hardware Bring-Up

1. **Synthesize** → **Implement** → **Generate Bitstream** in Vivado.
2. Program the Nexys A7.
3. Demo script (record on video):
4. Show raw word on SW15-SW0.

5. Cycle SHAMT through 0...15 via BTNC; observe rotating digits.
6. Toggle DIR and ROT while holding a visible pattern (e.g., 0xFE01).
7. Point out LED read-outs.

Submission (identical format to Lab 6)

```
git add Lab7/*
git commit -m "ECE3300L Lab7: Barrel Shifter & Display"
git push origin main
```

Include:

- **Verilog sources:** clock_divider_fixed.v, debounce_toggle.v, barrel_shifter16.v, shamt_counter.v, hex_to_7seg.v, seg7_scan8.v, top_lab7.v.
- **Testbenches** (*_tb.v).
- **PDF report** (design explanation, simulations, hardware photos, block diagram).
- **Demo video link** (YouTube or unlisted Drive).

Grading (same weights as Lab 5 & 6)

Item	Weight
Demo Video	50 %
Testbenches / Simulation	25 %
Written Report (PDF)	25 %

⚠ Missing any single deliverable = **zero** for that section!

Questions?

Ask early on Microsoft Teams. Keep the Verilog *modular*, keep the demo *clear*, and happy shifting!